

Spaced Repetition as a Cure to “Coded Bias”

Candidate code: lrr955

Page count: 30

Introduction

One of the greatest concerns in the field of machine learning is that of “Coded Bias,” named after a 2020 documentary exploring the subject, where models develop their own prejudice because of skewed training datasets. Among these models are facial recognition algorithms, which are less accurate in recognizing faces of women and racial minorities due to pictures of those groups being underrepresented in both the datasets and on the internet. This inaccuracy is preventable, as shown by MIT Media Lab researcher Joy Buolamwini; by simply adding more pictures of women and racial minorities to the model’s training dataset, Buolamwini successfully trained a model that recognized faces from those groups as accurately as White and male faces (Kantayya, 2020).

Although this technique is effective, I am disturbed by the amount of subjective interpretation it relies on. For example, aiming for equal representation of ethnic groups in training data begs the question of whether to consider Chinese, Japanese, and Koreans as separate groups or as a single “East Asian” group. By tasking humans to make these distinctions, we reintroduce the biases that Buolamwini sought to eliminate. Furthermore, fixating on race and sex risks underrepresenting other minorities and outliers in the training data, consequently creating new biases. Ultimately, humans are horribly ineffective at accounting for every possible minority in a dataset. All these factors culminate towards further divisions and hostility about equality and social justice, a result that my people-pleasing mindset will not tolerate. For this reason, and because I have experience in

programming, I seek to create a machine learning algorithm that resolves “Coded Bias” without subjective human intervention.

An Introduction to Machine Learning and Models

Understanding the emergence of “Coded Bias” requires understanding how machine learning algorithms like facial recognition find patterns to begin with. Note that the end goal of machine learning is to create a “model,” which is like an engineered, digital brain that can read an input and respond with the correct output. For example, a facial recognition model should output “yes” upon receiving an image of a face and “no” upon receiving an image that is not a face.

Machine learning models often learn by iterating over training data sets, which are big collections of possible inputs, each paired with their correct output. In the case of facial recognition models, this means thousands of images, each with a label identifying them as a “face” or “not a face.” Models learn by using their training data sets like a deck of flashcards; they read an input from the training data, they predict an output, and they compare their prediction with the correct output. Each time they do so, they alter their internal thinking to better match the correct output, a process I will term “fitting.” After repeating this process for every element in the training data multiple times, the model’s accuracy should increase to a desired level.

In the case where a particular group is unrepresented, they will have a minimal effect on the model despite mismatches between predictions and correct outputs. This is

because the model iterates over members of that minority very few times. As a result, the neural network is poorly fitted to the minority group.

Spaced Repetition: Machine-Friendly or Humans-Only?

My solution to “Coded Bias” draws inspiration from Anki and Remnote, which are flashcard programs that I use to review a variety of topics including biology and world history. Anki and Remnote are unique in their usage of spaced repetition algorithms, which are variations of typical flashcard algorithms; whereas a typical flashcard algorithm would repeatedly test me on each card at equal frequencies, spaced repetition algorithms identify and target my weaknesses by showing some cards at higher frequencies than other cards. To find cards I am weak at, they take each card and count the number of times my predicted answer is incorrect for that card. Then, they mark the cards with the most incorrect predictions as “difficult,” and show those cards more often than “easy” cards. This will help me memorize even the “difficult” cards.

Seeing how widely spaced repetition can be applied, I am determined to try creating a machine learning algorithm that uses spaced repetition to train a model. In doing so, I hope that inputs from minority groups would initially be marked “difficult,” causing the model to train on them more often. This would counteract the “Coded Bias” against minority groups by ensuring equal accuracy for all inputs– all without relying on subjective human intervention.

Admittedly, most spaced repetition algorithms such as Anki’s Free Spaced Repetition Scheduler (FSRS) and SuperMemo2 (SM-2) are designed for retention in human

memory, not in machine learning models (Bjornstad, 2024; Ye, 2025a). As a result, these algorithms may prove ineffective at counteracting “Coded Bias.” To test their efficacy, I will use hypothesis testing to answer the following inquiry: Are models from spaced repetition training significantly more accurate than models from traditional training methods at recognizing minority groups?

The Chi-Square Hypothesis Test

To find the accuracy of a model, I simply give it a testing dataset of items and count the number of correct predictions the model makes. This number can be compared between models to decide between two hypotheses: The Null Hypothesis H_0 states that spaced repetition training causes no significant improvement in recognizing minority groups, and that any differences between their accuracy are coincidental. If this is true, we would expect models from spaced repetition training and traditional training to have about the same accuracy numbers in recognizing minority groups. If, on the contrary, models from spaced repetition training have much greater accuracy in recognizing minority groups, then the chance of H_0 being true becomes very low. Thus, we reject H_0 in favor of the Alternative Hypothesis H_1 , which states that spaced repetition does cause a significant improvement.

Instead of subjectively deciding whether the difference is great enough to reject H_0 , we use hypothesis testing to make that decision. Hypothesis testing helps to find the p-value, which is the probability of measuring our data if H_0 is true. If this p-value is too small, usually below 0.05, then we reject H_0 in favor of H_1 .

There are multiple ways of finding the p-value, including the Z-test, T-test, and chi-square test (Jain, 2019). Out of these, the chi-square hypothesis test is best for comparing the accuracy frequencies of models, because they are discrete, counted numbers (Kashif, 2024). For our purposes, the chi-square process is as follows: First, identify two disjoint cases to compare, such as a model trained with regular methods R and a model trained with spaced repetition (S.R.) R' . Second, count the correct predictions C and incorrect predictions C' for both cases, yielding 4 total disjoint cases $R \cap C$, $R' \cap C$, $R \cap C'$, and $R' \cap C'$ (where $R \cap C$ denotes the case where the regular model correctly predicts an image of a face to be a face, or an image without a face to lack a face). We can arrange the observed frequencies O of each set below:

	Correct Predictions C	Incorrect Predictions C'	Total U
Regular Model R	$O_{R \cap C}$	$O_{R \cap C'}$	O_R
S.R. Model R'	$O_{R' \cap C}$	$O_{R' \cap C'}$	$O_{R'}$
Total U	O_C	$O_{C'}$	O_U

Table arrangement comparing majority and minority faces with correct and incorrect predictions. U is the union (total) set.

Third, use the values of O_R , $O_{R'}$, O_C , $O_{C'}$, and O_U to find E , the expected frequency when H_0 is assumed, for each of my 4 disjoint cases. This process is shown in the section titled “The Negative Control.”

Fourth, find a number χ^2 , which will indicate the level of discrepancy between the observed and expected frequencies. χ^2 is found by using the following formula, repeated for each of the 4 disjoint cases i :

$$\chi^2 = \sum \frac{(O_i - E_i)^2}{E_i}$$

Fifth, find a number called the degrees of freedom df . This process is shown in the section titled “The Negative Control.”

Sixth, using χ^2 and df , we can find whether the p-value exceeds 0.05 by comparing χ^2 to the corresponding number on the table below:

df	1	2	3	4	5	6	7	8
χ^2 comparand	3.841	5.991	7.815	9.488	11.070	12.592	14.067	15.507

Given the degrees of freedom df , the p-value approximately equals 0.05 when χ^2 equals the corresponding comparand listed above (Frost, 2022).

If χ^2 is greater than said number, then the p-value is less than 0.05, so we reject H_0 . Otherwise, the p-value is greater than 0.05, so we cannot reject H_0 . In our example, rejecting H_0 means that there is a difference in facial recognition accuracy between a regular and S.R. trained model, and this difference is large enough that the probability of it appearing under H_0 is less than 0.05. Although it is impossible to reduce said probability to 0 (leaving possibility for error), 0.05 is the accepted threshold to reject H_0 (Jain, 2019).

The Negative Control: Creating “Coded Bias”

Instead of recognizing faces, the models I will use in my experiments will distinguish pictures of handwritten digits from the MNIST dataset, which is both standardized and simpler than datasets used for facial recognition (Deng, 2012). This allows for faster training and testing. To prove whether spaced repetition is effective for eliminating “Coded Bias” in digit recognition, I must first show that “Coded Bias” exists to begin with. To do this, I will emulate Buolamwini’s situation with her facial recognition model; I will arbitrarily select two digits to be the majority (M) and minority (M') groups and compose a training dataset with the majority digit group, minority digit group, and additional filler pictures. Afterward, the model will be trained (using traditional methods) to recognize filler pictures from non-filler pictures. The table below outlines this analogy:

Buolamwini’s facial recognition model	Experiment model	Example
Images of White and male faces (majority of faces in training dataset)	Majority digit group	5421 images of “0”
Facial images from women and people of color (minority of faces in training dataset)	Minority digit group	2711 images of “4”
Non-Facial Images	Filler pictures of remaining digits	8132 images of digits besides “0” and “4”

The analogy above depicts how my setup (right) will emulate “Coded Bias” seen in

Buolamwini’s facial recognition model (left).

Let C represent the case where a majority or minority digit is correctly recognized as non-filler and let C' represent the opposite case. If done correctly, the resulting model will misrecognize minority digit images more often than it does with majority digit images; in other words, $P(C' | M') > P(C' | M)$.

This can be verified with a custom testing dataset. Note that the model needs to be trained and tested twice, the second time with the majority and minority digits swapped, to ensure that the shape of each digit does not affect the accuracy of the majority and minority groups being recognized. Below are the results of one such test:

	Correctly Recognized C	Misrecognized C'	Total U
Majority Digit M	1873	89	1962
Minority Digit M'	1740	222	1962
Total U	3613	311	3924

The number of each cell in the table above represents O_i for the intersection of its corresponding row and column. For example, $O_{M \cap C} = 1873$.

Let our Null Hypothesis H_0 state that M and C are independent, and thus there is no significant difference between the models' accuracy in recognizing majority and minority digit images. The chi-square test requires knowing E_i , the expected frequencies of each of the 4 disjoint cases $M \cap C$, $M' \cap C$, $M \cap C'$, and $M' \cap C'$, assuming H_0 is true. We solve for E_i using the principle that $P(A \cap B) = P(A) \times P(B)$ if A and B are independent events.

Since H_0 states that M and C are independent, we can estimate the expected number of majority digits recognized correctly ($E_{M \cap C}$) like so:

$$P(M) = \frac{O_M}{O_U}$$

$$P(C) = \frac{O_C}{O_U}$$

$$P(M \cap C) = P(M) \times P(C) = \frac{O_M \times O_C}{O_U^2}$$

$$E_{M \cap C} = P(M \cap C) \times O_U = \frac{O_M \times O_C}{O_U} = \frac{1962 \times 3613}{3924} \approx 1807$$

Notice how this process ends in an expression using O_M , O_C , and O_U , which can be applied as a formula for the 3 other cases by exchanging M with M' and C with C' . Doing so gives us the following expected frequencies for the 4 disjoint cases:

$$E_{M \cap C} \approx 1807$$

$$E_{M \cap C'} = \frac{O_M \times O_{C'}}{O_U} = \frac{1962 \times 311}{3924} \approx 155.5$$

$$E_{M' \cap C} = \frac{O_{M'} \times O_C}{O_U} = \frac{1962 \times 3613}{3924} \approx 1807$$

$$E_{M' \cap C'} = \frac{O_{M'} \times O_{C'}}{O_U} = \frac{1962 \times 311}{3924} \approx 155.5$$

We can substitute these values and the observed frequencies in the table above into the chi-square formula to find χ^2 , rounded the thousandths place to match the precision of its comparand:

$$\begin{aligned} \chi^2 &= \sum \frac{(O_i - E_i)^2}{E_i} \\ &= \frac{(O_{M \cap C} - E_{M \cap C})^2}{E_{M \cap C}} + \frac{(O_{M \cap C'} - E_{M \cap C'})^2}{E_{M \cap C'}} + \frac{(O_{M' \cap C} - E_{M' \cap C})^2}{E_{M' \cap C}} + \frac{(O_{M' \cap C'} - E_{M' \cap C'})^2}{E_{M' \cap C'}} \end{aligned}$$

$$= \frac{(1873 - 1807)^2}{1807} + \frac{(89 - 155.5)^2}{155.5} + \frac{(1740 - 1807)^2}{1807} + \frac{(222 - 155.5)^2}{155.5}$$

$$\approx 61.774$$

61.774 is a large value for χ^2 , but we still need to find the degrees of freedom df to find the comparand for χ^2 . df is equal to the possible states of the independent variable minus 1, all multiplied by the possible states of the dependent variable minus 1 (Kashif, 2024). Our testing dataset has 2 possible states for the digit group, M and M' , and 2 possible states for the accuracy, C and C' , so $df = 1 \times 1 = 1$. Therefore:

$$\chi^2 > 3.841 \Rightarrow p < 0.05$$

$$\chi^2 \approx 61.774 > 3.841$$

$$\therefore p < 0.05$$

Since the p-value, or the probability of our data appearing under H_0 , is way less than 0.05, we reject H_0 and conclude that the model is more likely to confuse minority digits with filler images than majority digits. The next step is to test whether using spaced repetition in training significantly reduces this “coded bias.”

Adding Spaced Repetition

Next, I compared the effectiveness of a traditionally trained “Regular” model, a model trained with FSRS, and a model trained with SM-2 at recognizing the minority digits (M'). I programmed these models using public libraries imported into Python (Ye, 2025b; Hamilton, 2024; Chollet, 2019). My code is in Appendix B. Like in “The Negative Control,” I counted the number of times each type of model correctly recognized (C) and misrecognized (C') minority digit images. After noticing that both spaced repetition models have accuracy values that range widely, I repeated this experiment 10 times and accumulated the results to minimize the effect of sampling error.

Next, I used the chi-square test to see if the spaced-repetition models performed significantly better than the Regular model. The SM-2 models had lower accuracy values than the Regular models, so we cannot conclude that they are more accurate; we can ignore them. The FSRS (R') models had slightly higher accuracy compared to the Regular (R) models, shown below:

	Correct Recognition C	Misrecognition C'	Total M'
Regular models R	8241	1579	9820
FSRS models R'	8434	1386	9820
Total M'	16675	2965	19640

The number of each cell in the table above is O_i for the intersection of its corresponding row and column. For example, $O_{R \cap C} = 8241$. Note that M' acts as the total, or union, because the testing dataset consists of only minority digits.

For the chi-square test, let the Null Hypothesis H_0 state that R and C are independent, and that FSRS models are no more accurate at recognizing minority digits than Regular models. Under H_0 , the expected frequencies E_i for the 4 disjoint cases $R \cap C$, $R' \cap C$, $R \cap C'$, and $R' \cap C'$ can be found by substituting the row and column totals O_R , $O_{R'}$, O_C , $O_{C'}$, and $O_{M'}$ into the formula like so:

$$E_{R \cap C} = \frac{O_R \times O_C}{O_{M'}} = \frac{9820 \times 16675}{19640} = 8337.5$$

$$E_{R \cap C'} = \frac{O_R \times O_{C'}}{O_{M'}} = \frac{9820 \times 2965}{19640} = 1482.5$$

$$E_{R' \cap C} = \frac{O_{R'} \times O_C}{O_{M'}} = \frac{9820 \times 16675}{19640} = 8337.5$$

$$E_{R' \cap C'} = \frac{O_{R'} \times O_{C'}}{O_{M'}} = \frac{9820 \times 2965}{19640} = 1482.5$$

Next, we substitute our observed and expected frequencies for the 4 disjoint cases into the chi-square formula to find χ^2 , which equals 14.797.

Since our independent and dependent variable both have 2 possible states (R vs. R' and C vs. C'), $df = 1 \times 1 = 1$. Thus:

$$\chi^2 > 3.841 \Rightarrow p < 0.05$$

$$\chi^2 \approx 14.797 > 3.841$$

$$\therefore p < 0.05$$

Since the chances of our data occurring if R and C are independent has a probability of less than 5%, we reject H_0 and confirm that using FSRS in the training algorithm improves a machine learning model's ability to recognize minority digits. Note that, while

the difference in accuracy between FSRS and Regular models is statistically “significant,” this does not translate to the human understanding of “significant.” In this situation, rejecting H_0 merely implies that the higher accuracy of FSRS models is not a coincidence. However, it could still be considered small; when it comes to recognizing minority digit images, my FSRS models had less than a 12.3% decrease in misrecognition compared to Regular models.

Drawbacks of Spaced Repetition

Such a small improvement may not be worth the drawbacks of using FSRS to train models. First, since spaced repetition training requires the usage of priority queues, which are slower than typical data structures used for machine learning, the training time is almost an order of magnitude longer than traditional training methods. Second, while the FSRS models were better at recognizing minority digit images, their overall accuracy for both majority and minority digit images is worse than that of Regular models. This is most likely caused by the FSRS training process “fixating” on minority digit images and neglecting majority digit images.

To decide whether the benefit of using spaced repetition outweighs the drawbacks, I first must optimize my spaced repetition algorithm to maximize its benefits and minimize its drawbacks. This way, if spaced repetition training turns out to be ineffective, I can be certain with that conclusion.

Extension: Optimizing Spaced Repetition

My first step in optimizing my spaced repetition-trained models is to use the FSRS built-in optimizer, which modifies the FSRS algorithm to better fit my machine learning models (Ye, 2025b). Next, I tried optimizing the training process' other primary variable: the batch size. For each batch size (b_i), which were powers of 2 from 32 to 512, I trained 10 models with the newly optimized FSRS, tested each of them on my custom testing dataset, and added up the number of misrecognitions (C') for the majority (M) and minority (M') digit images. I ended with the following results:

Batch size b_i (items per batch)	Summed number of misrecognitions across 10 models		
	Majority digit "0": $O_{M \cap C'}$	Minority digit "4": $O_{M' \cap C'}$	Total: $O_{C'}$
32	2314	1663	3977
64	2093	1088	3181
128	685	1560	2245
256	1148	3060	4208
512	2712	4168	6880

Frequencies of misrecognition made by FSRS models trained with varying batch size b_i .

Two features are notable with this data. The first is that there is a value for b_i between 256 and 64 that minimizes misrecognitions ($O_{C'}$). The second is more subtle: For batch sizes 32 and 64, the model misrecognized more majority digits than minority digits ($O_{M \cap C'} > O_{M' \cap C'}$), but for batch sizes 128 to 512, the opposite was true ($O_{M \cap C'} < O_{M' \cap C'}$). Knowing this, I can interpolate that there exists a batch size b_Ω between 64 and 128 that acts as a "turning point" such that the following are true:

$$b_i < b_\Omega \Leftrightarrow O_{M \cap C'} > O_{M' \cap C'}$$

$$b_i > b_\Omega \Leftrightarrow O_{M \cap C'} < O_{M' \cap C'}$$

$$b_i \approx b_\Omega \Leftrightarrow O_{M \cap C'} \approx O_{M' \cap C'}$$

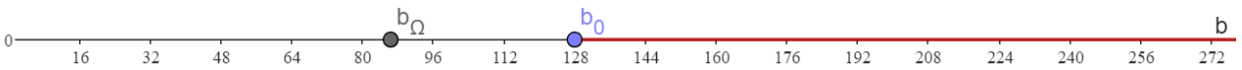
This feature, called monotonicity, is crucial for our next step in optimizing batch size.

Binary Searching for Optimal Batch Size

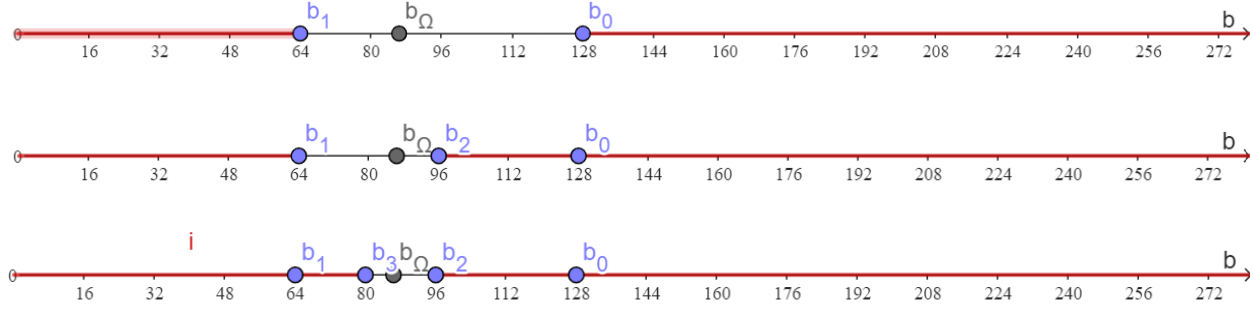
Since equal accuracy is the goal of using spaced repetition training, b_Ω would be the perfect batch size for use in my FSRs models. Although I cannot calculate it directly with the limited, imprecise data I currently have, the monotonic character of batch size provides a relatively fast method of guessing and checking for it, known by programmers like myself as binary search. We can visualize this with a number line for all values of b . Note that, while the value b_Ω is plotted on the number line for convenience, it is invisible to us because we do not know it.



The first step is to choose an initial value b_0 , which I set to be 128, and compare the resulting values for $O_{M \cap C'}$ and $O_{M' \cap C'}$. We can use this comparison to infer whether $b_0 < b_\Omega$ or $b_0 > b_\Omega$. In this case, we get $O_{M \cap C'} < O_{M' \cap C'}$, so $b_0 > b_\Omega$. Thus, we can cut all values of b greater than b_0 , as they cannot be b_Ω :

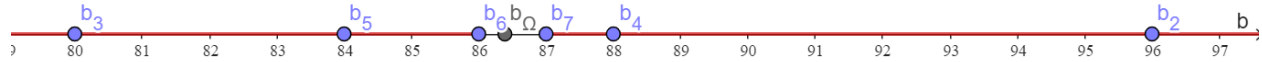


Next, we repeat this process with another value b_1 . Note that, as far as we know, $0 < b_\Omega < 128$. Choosing b_1 to be the midpoint of that interval is best since it guarantees cutting half of the interval. Thus, $b_1 = \frac{0+128}{2} = 64$. Then, we can repeat the entire sequence like so:



After about 6 or 7 iterations, we would ideally reach a value b_i where $b_i = b_\Omega$.

However, as shown below, it's likely that we end with values $b_6 = b_7 - 1$ where $O_{M \cap C'} > O_{M' \cap C'}$ for b_6 and $O_{M \cap C'} < O_{M' \cap C'}$ for b_7 . This is because there is no rule stating that there even exists a value $b_\Omega \in \mathbb{Z}^+$ that causes our models to be perfectly balanced.



Knowing this, we can stop searching and conclude that b_6 and b_7 will be as close as we can get to achieving equal accuracy for M and M' . Afterward, we can decide between using b_6 and b_7 for our models based on other metrics, such as O_C .

Implementation

To account for random sampling errors in the values for $O_{M \cap C'}$ and $O_{M' \cap C'}$, the experiment used several redundancies that differed from traditional binary search. First, instead of training 10 models for each value of b_i , I trained 20 models to reduce sampling

error. This lowers the probability of a “binary search error” where $O_{M \cap C'}$ being coincidentally greater or lesser than $O_{M' \cap C'}$ gives us the wrong information about b_Ω .

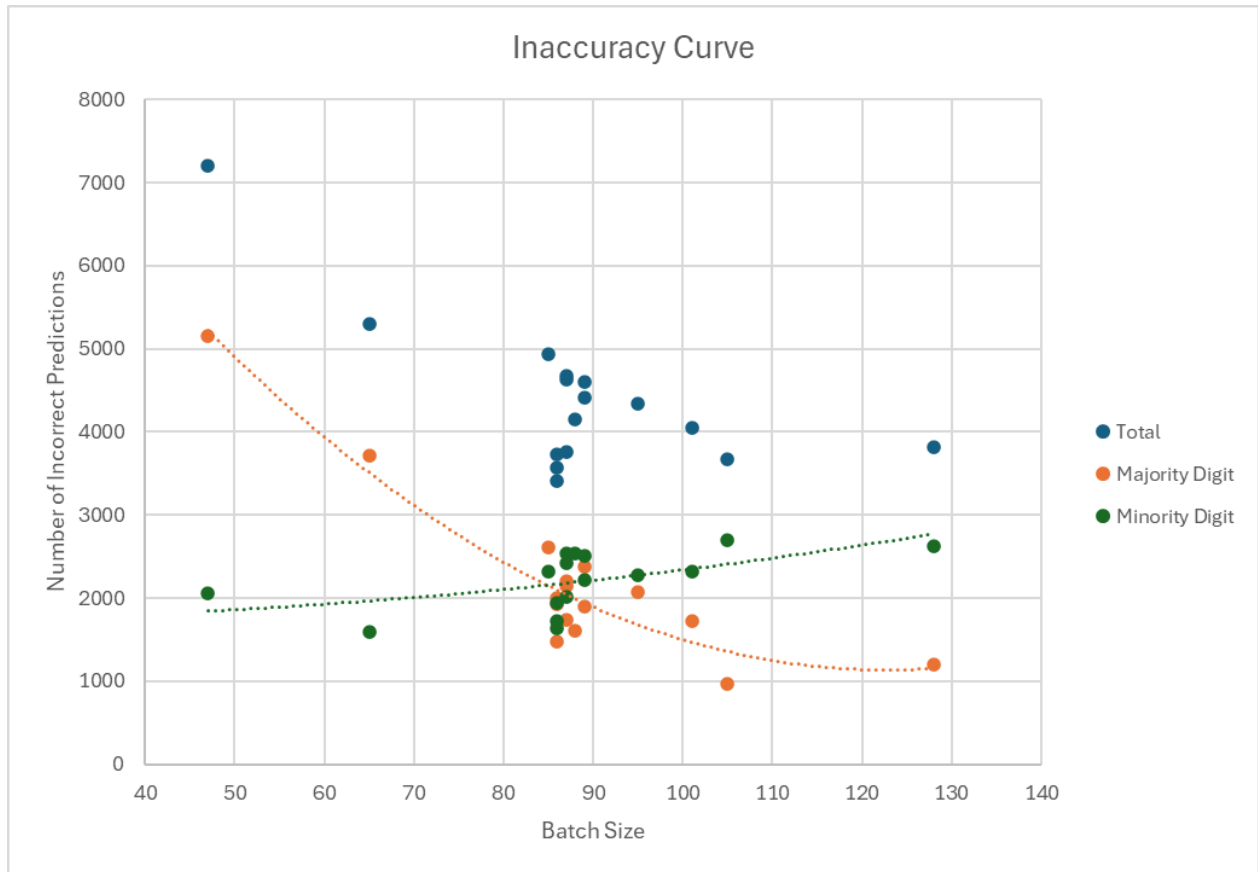
In case a “binary search error” still happens, I changed the rate at which the jump distance $|b_{i+1} - b_i|$ decreases. Notice that, in traditional binary search, the jump distance between b_i and b_{i+1} decreases exponentially, under powers of $\frac{1}{2}$. This helps to alter the value of b_i quickly at first, before switching to a slower, more precise search the closer b_i becomes to b_Ω . However, it also prevents us from ever testing values of b_i that have previously been eliminated, which is a problem if a “binary search error” occurs. I changed the rate of decrease to be under powers of $\frac{2}{3}$, slowing the decrease and thus allowing the experiment to consider values for b_i it would have otherwise eliminated outright.

I stopped the search program upon noticing that b_i was mostly fluctuating between 86 and 87, suggesting that b_Ω was between them. The following table lists each value for b_i and their corresponding values for $O_{M \cap C'}$, $O_{M' \cap C'}$, and $O_{C'}$, in the order they were tested:

Batch size b_i (items per batch)	Summed number of misrecognitions across 20 models		
	Majority digit "0": $O_{M \cap C'}$	Minority digit "4": $O_{M' \cap C'}$	Total: $O_{C'}$
128	1195	2624	3819
47	5149	2061	7210
101	1727	2322	4049
65	3716	1590	5306
89	2385	2216	4601
105	970	2697	3667
95	2069	2273	4342
89	1906	2506	4412
85	2616	2328	4944
87	2146	2536	4682
86	1932	1642	3574
87	2212	2420	4632
86	2004	1730	3734
88	1615	2533	4148
87	1733	2020	3753
86	1472	1944	3416

The results of binary searching for b_Ω for my FSRs models, ordered by $i = 0,1,2,3 \dots 16$.

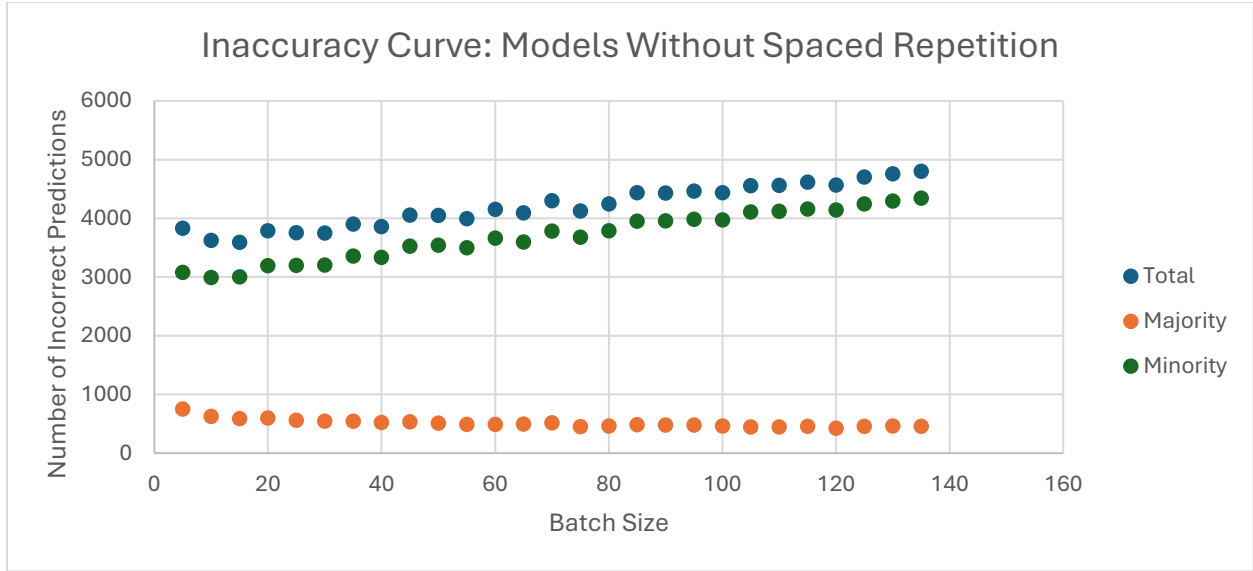
Since $b_i = 86$ yields more accurate results than $b_i = 87$ overall, I chose 86 as the optimal batch size for my FSRs training process. Below is a graph of the above data:



Note that the trendlines for inaccuracy rates for Majority and Minority digits intersect near $b = 85$, supporting the optimal batch size being 86.

Comparison to Regular Models

Lastly, to judge whether using FSRS is worth its cost, I need to find the optimal batch size for Regular models. To do this, I trained and tested Regular models using batch sizes from 5 to 135 (at intervals of 5) and found the one that results in the highest accuracy. This was not possible with FSRS models due to their slow training time. A resulting data table is in Appendix A. Below is the graph of this data:



Based on the data, I set the optimal batch size of Regular models to be $b_i = 15$.

When both FSRS and Regular models are optimized at their respective batch sizes, their performance is about the same, as demonstrated by the data below:

	Correct Recognitions C	Misrecognitions C'	Total U
Regular models at optimal batch size R	35647	3593	39240
FSRS models at optimal batch size R'	35666	3574	39240
Total U	71313	7167	78480

The number of each cell in the table above represents O_i for the intersection of its corresponding

row and column. For example, $O_{R \cap C} = 35647$.

Another chi-square test comparing the accuracies of Regular and FSRS models at their optimal batch size outputs $\chi^2 = 0.055$. Since $df = 1$ and $\chi^2 < 3.841$, the p-value is greater than 0.05. Since the chances of my data occurring by coincidence is greater than

5%, we cannot reject the Null Hypothesis H_0 , which states that the Regular and FSRS models have no significant difference in overall accuracy.

A limitation to this result is that, although we cannot reject H_0 , it is not necessarily true. This is because we have no way of rejecting the Alternative Hypothesis H_1 , which states that the FSRS models are either more accurate or less accurate than the Regular models overall. To demonstrate this, consider the scenario where H_1 is true, yet the FSRS models performed about as accurately as the Regular models by random chance. Since we have no way of eliminating this scenario, we must acknowledge that H_1 is plausible. Nonetheless, we should further investigate FSRS models until we rule out their viability.

Not only do the optimized FSRS models perform about as well as optimized Regular models in terms of overall accuracy, but they are much better in recognizing images of minority digits. In this category, optimized Regular models made a total of 3003 misrecognitions, while optimized FSRS models made 1642 misrecognitions for a 45.3% decrease. Ultimately, the optimized FSRS models are just as accurate in recognizing minority digit images as majority digit images, eliminating the “coded bias” present in Regular models.

Conclusion

In this investigation, I successfully eliminated “Coded Bias,” the phenomenon where underrepresented groups are recognized less accurately than overrepresented groups, in digit image recognition models, all whilst avoiding subjective human intervention. I did this by using the Free Spaced Repetition Scheduler (FSRS) to train the models. At the same time, using FSRS results in no significant decrease in the model’s overall accuracy. The main drawback of using FSRS is time; my process of optimizing the FSRS training parameters, particularly batch size, took hours to complete, in contrast with the few minutes required for traditional training. Note, however, that this slowdown is primarily caused by my limited programming knowledge, and my basic FSRS models could be made faster in the future. Overall, the existing benefits of using spaced repetition demonstrate that it is worth researching to mitigate its weaknesses.

References

- Bjornstad, S. (2024). *The Anki SM-2 spaced repetition algorithm*. RemNote.
<https://help.remnote.com/en/articles/6026144-the-anki-sm-2-spaced-repetition-algorithm>
- Chollet, F. (2019, March 1). *Writing a training loop from scratch in TensorFlow*. Keras.io.
https://keras.io/guides/writing_a_custom_training_loop_in_tensorflow/
- Deng, L. (2012). The MNIST database of handwritten digit images for machine learning research. *IEEE Signal Processing Magazine*, 29(6), 141–142.
<https://doi.org/10.1109/msp.2012.2211477>. Accessed MNIST dataset only.
- Frost, J. (2022, January 26). *Chi-Square Table*. Statistics by Jim.
<https://statisticsbyjim.com/hypothesis-testing/chi-square-table/>
- Hamilton, J. (2024, December 7). *anki-sm-2*. PyPI. <https://pypi.org/project/anki-sm-2/>. GNU Affero General Public License v3.
- Jain, D. (2019, October 13). *Hypothesis testing*. GeeksforGeeks.
<https://www.geeksforgeeks.org/software-testing/understanding-hypothesis-testing/>
- Kantayya, S. (Director). (2020, November 11). *Coded bias* [Film]. 7th Empire Media.
- Kashif, M. (2024, May 3). *Chi-square test*. GeeksforGeeks.
<https://www.geeksforgeeks.org/maths/chi-square-test/>
- Ye, J. (2025a, September 23). *ABC of FSRS*. GitHub. <https://github.com/open-spaced-repetition/fsrs4anki/wiki/ABC-of-FSRS>
- Ye, J. (2025b, October 13). *fsrs*. PyPI. <https://pypi.org/project/fsrs/>. MIT License.

Appendix A

Optimal Batch Size Search Results (Regular models)

Batch size b_i (items per batch)	Summed number of misrecognitions across 20 models		
	Majority digit “0”: $O_{MnC'}$	Minority digit “4”: $O_{M'nC'}$	Total: $O_{C'}$
5	749	3080	3829
10	628	2994	3622
15	590	3003	3593
20	596	3191	3787
25	559	3196	3755
30	542	3205	3747
35	544	3356	3900
40	521	3337	3858
45	531	3526	4057
50	510	3540	4050
55	491	3501	3992
60	488	3664	4152
65	496	3598	4094
70	517	3781	4298
75	449	3678	4127
80	461	3786	4247
85	486	3949	4435
90	478	3954	4432
95	479	3986	4465
100	465	3970	4435
105	447	4110	4557
110	446	4118	4564
115	458	4159	4617
120	425	4143	4568
125	458	4246	4704
130	465	4294	4759
135	458	4342	4800

Appendix B

```

from keras.models import Sequential
import tensorflow as tf
import fsrs
import anki_sm_2
import heapq

```

```

import numpy as np
class Spaced_Repetition_Train_Card:
    def __init__(self, x, y, card, id=0):
        self.id = id
        self.x = x
        self.y = y
        self.card = card
    def __lt__(self, other):
        return self.card.due < other.card.due

class FSRS(Sequential):
    def __init__(self, layers):
        super().__init__(layers)
        self.scheduler = fsrs.Scheduler()
        self.review_logs : list[fsrs.ReviewLog] = []
        self.debug = False

    def train_step(self, data):
        batch_x, batch_y, batch_card = data
        batch_size = len(batch_x)

        with tf.GradientTape() as tape:
            y_predict = self(batch_x, training=True)
            loss = self.compute_loss(y=batch_y, y_pred=y_predict)

            quality = self.rate_accuracy(y=batch_y, y_pred=y_predict)
            review_logs = np.array([None] * batch_size)

            for i in range(batch_size):
                batch_card[i], review_logs[i] =
self.scheduler.review_card(batch_card[i], quality[i]) # type: ignore
                heapq.heappush(self.pq, \
Spaced_Repetition_Train_Card(batch_x[i], batch_y[i], batch_card[i]))

            trainable_vars = self.trainable_variables
            gradients = tape.gradient(loss, trainable_vars)
            self.optimizer.apply(gradients, trainable_vars)

        self.review_logs += review_logs

    for metric in self.metrics:
        if metric.name == "loss":
            metric.update_state(loss)
        else:
            metric.update_state(batch_y, y_predict)

```

```

        return {m.name: m.result() for m in self.metrics}

    def fit(self, x, y, epochs: int=1, batch_size: int=32):
        #set up min heap
        self.pq = []#min heap that puts the first due training card \
at the front.
        for id, [xi, yi] in enumerate(zip(x, y)):
            heapq.heappush(self.pq, Spaced_Repetition_Train_Card(xi,\
yi, card=fsrs.Card(), id=id))

        for epoch in range(epochs):
            #print(f"\nStart of epoch {epoch}")
            stepsPerEpoch: int = len(x) // batch_size + 1
            for stepID in range(stepsPerEpoch):
                ##put cards into batches
                batch_x = np.empty((batch_size, 28, 28))
                batch_y = np.empty(batch_size)
                batch_card = [fsrs.Card()] * batch_size
                if self.debug:
                    print(f"Batch {stepID} Contents: ")
                for i in range(batch_size):
                    train_card = heapq.heappop(self.pq)
                    batch_x[i] = train_card.x
                    batch_y[i] = train_card.y
                    batch_card[i] = train_card.card
                    if self.debug:
                        print(train_card.id, end=" ")
                if self.debug:
                    print()

                results = self.train_step((batch_x, batch_y,\
batch_card))

            for metric in self.metrics:
                metric.reset_state()

    def rate_accuracy(self, y, y_pred):
        pred = np.round(y_pred, decimals=0) #For Binary decision
        ret = [0] * len(y)
        for i in range(len(y)):
            if pred[i] == y[i]:
                ret[i] = fsrs.Rating.Good
            else:
                ret[i] = fsrs.Rating.Hard
        return ret

```

```

class SM2(Sequential):
    def __init__(self, layers):
        super().__init__(layers)
        self.scheduler = anki_sm_2.Scheduler()
        self.debug = False

    def train_step(self, data):
        batch_x, batch_y, batch_card = data
        batch_size = len(batch_x)

        with tf.GradientTape() as tape:
            y_predict = self(batch_x, training=True)
            loss = self.compute_loss(y=batch_y, y_pred=y_predict)

        quality = self.rate_accuracy(y=batch_y, y_pred=y_predict)
        review_logs = np.array([None] * batch_size)

        for i in range(batch_size):
            batch_card[i], review_logs[i] =
self.scheduler.review_card(batch_card[i], quality[i]) # type: ignore
            heapq.heappush(self.pq, \
Spaced_Repetition_Train_Card(batch_x[i], batch_y[i], batch_card[i]))

        trainable_vars = self.trainable_variables
        gradients = tape.gradient(loss, trainable_vars)
        self.optimizer.apply(gradients, trainable_vars)

        for metric in self.metrics:
            if metric.name == "loss":
                metric.update_state(loss)
            else:
                metric.update_state(batch_y, y_predict)

        return {m.name: m.result() for m in self.metrics}

    def fit(self, x, y, epochs: int=1, batch_size: int=32):
        #set up min heap
        self.pq = []#min heap that puts the first due training card \
at the front.
        for id, [xi, yi] in enumerate(zip(x, y)):
            heapq.heappush(self.pq, Spaced_Repetition_Train_Card(xi, \
yi, card=anki_sm_2.Card(), id=id))

        for epoch in range(epochs):
            print(f"\nStart of epoch {epoch}")
            stepsPerEpoch: int = len(x) // batch_size + 1

```

```

for stepID in range(stepsPerEpoch):
    ##put cards into batches
    batch_x = np.empty((batch_size, 28, 28))
    batch_y = np.empty(batch_size)
    batch_card = [anki_sm_2.Card()] * batch_size
    if self.debug:
        print(f"Batch {stepID} Contents: ")
    for i in range(batch_size):
        train_card = heapq.heappop(self.pq)
        batch_x[i] = train_card.x
        batch_y[i] = train_card.y
        batch_card[i] = train_card.card
        if self.debug:
            print(train_card.id, end=" ")
    if self.debug:
        print()

    results = self.train_step((batch_x, batch_y,
batch_card))

    for metric in self.metrics:
        metric.reset_state()

def rate_accuracy(self, y, y_pred):
    pred = np.round(y_pred, decimals=0)#For Binary decision
    ret = [0] * len(y)
    for i in range(len(y)):
        if pred[i] == y[i]:
            ret[i] = anki_sm_2.Rating.Good
        else:
            ret[i] = anki_sm_2.Rating.Hard
    return ret

```